LAPORAN PRAKTIKUM

POSTTEST (4)

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Ghina Cyntia Ramadhani (2509106054)

Kelas (B‘25)

PROGRAM STUDI INFORMATIKA

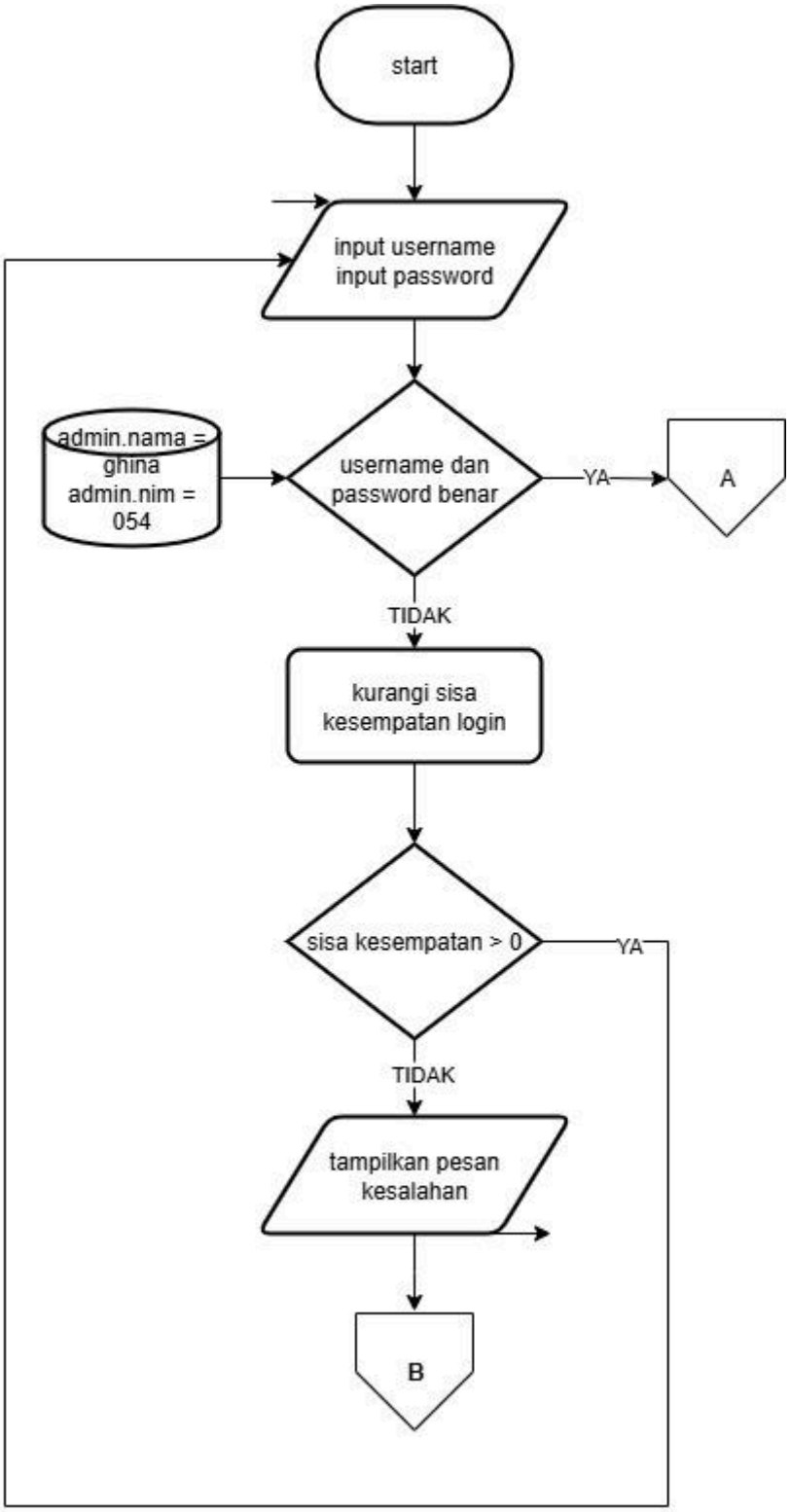
UNIVERSITAS MULAWARMAN

SAMARINDA

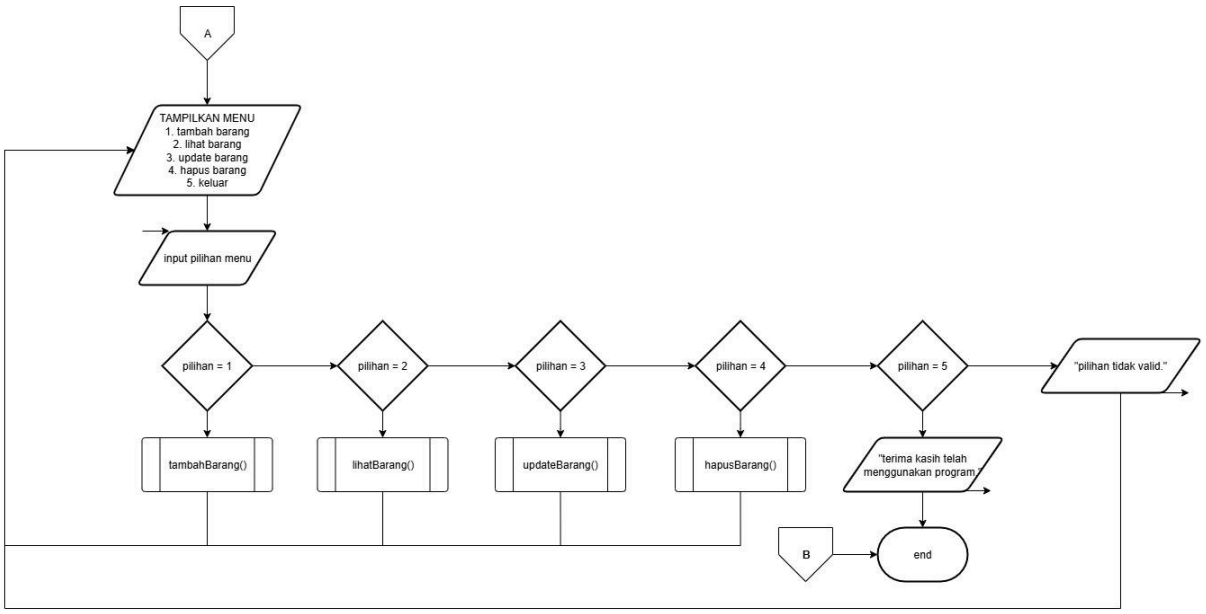
2026

1. Flowchart

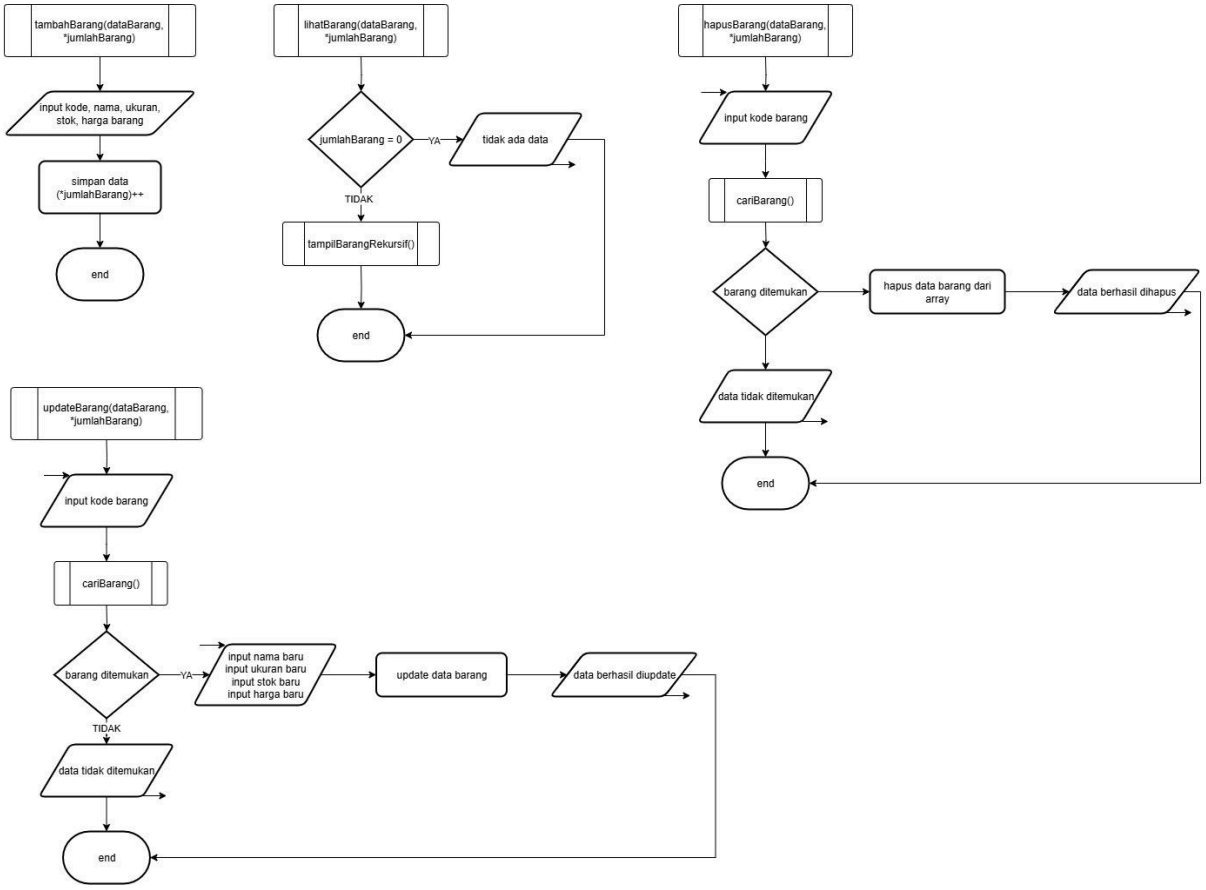
Program ini bertujuan untuk mengelola data barang seperti menambah, menampilkan, mengubah, dan menghapus data secara terstruktur. Pada program ini digunakan konsep pointer, di mana data seperti jumlah barang dikirim ke fungsi menggunakan address-of operator (&) dan diakses menggunakan dereference operator (*), serta pointer pada struct untuk mempermudah pengolahan data. Alur program dimulai dari proses login, kemudian masuk ke menu utama, pengguna memilih fitur yang diinginkan, sistem menjalankan subprogram sesuai pilihan, lalu kembali ke menu hingga pengguna memilih keluar. Program berakhir ketika pengguna memilih menu keluar atau gagal login setelah beberapa kali percobaan.



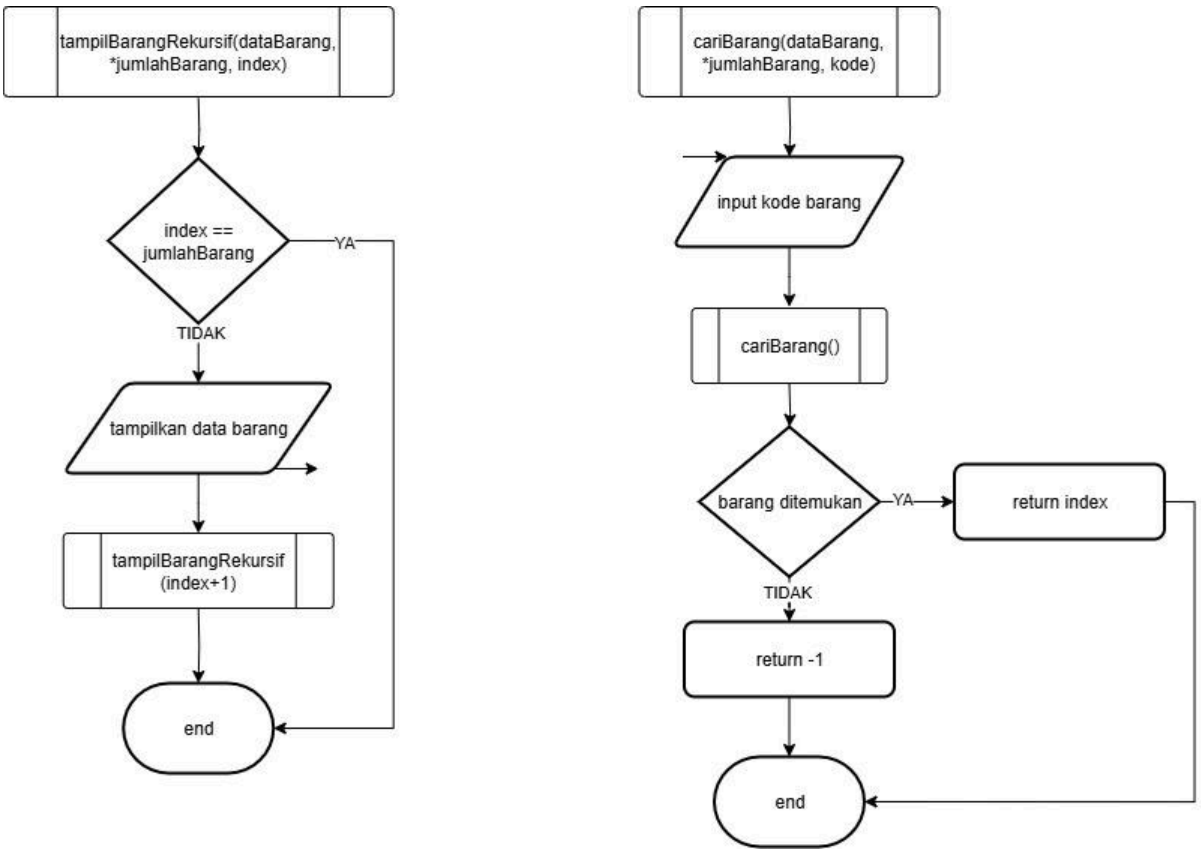
Gambar 1.1 Flowchart Login



Gambar 1.2 Flowchart Menu Utama



Gambar 1.3 Flowchart Prosedur



Gambar 1.4 Flowchart Fungsi

2. Deskripsi Singkat Program

Program ini bertujuan untuk membantu mengelola data barang pada sebuah toko pakaian secara sederhana dengan fitur CRUD (Create, Read, Update, dan Delete) untuk data barang. Program ini juga memiliki sistem login agar hanya pengguna tertentu yang dapat mengakses program. Dengan adanya program ini, proses pencatatan dan pengelolaan data barang dapat dilakukan dengan lebih mudah, cepat, dan terorganisir juga menghindari kesalahan pencatatan.

3. Source Code

A. Fitur Login

Fitur ini digunakan jika pengguna ingin login program.

Source Code:

```
//LOGIN
bool login(User *admin){

    string inputNama, inputNim;
    int sisa = 3;

    while(sisa > 0){
        system("cls");
        cout << "=== LOGIN ===\n";

        cout << "Username (Nama): ";
        cin >> inputNama;

        cout << "Password (NIM): ";
        cin >> inputNim;

        if(inputNama == admin->nama && inputNim == admin->nim){
            return true;
        } else {
            sisa--;
            cout << "Login gagal! Sisa kesempatan: " << sisa << endl;
            tekanEnter();
        }
    }
```

```
    return false;
}
```

B. Fitur Tampil Barang

Fitur ini digunakan untuk menampilkan data barang.

Source Code:

```
//TAMPIL BARANG
void tampilBarangRekursif(Barang dataBarang[], int *jumlahBarang, int index){

    if(index == *jumlahBarang){
        return;
    }

    Barang *ptr = &dataBarang[index];

    cout << left << setw(10) << ptr->kode
         << setw(20) << ptr->nama
         << setw(10) << ptr->detail.ukuran
         << setw(10) << ptr->stok
         << setw(15) << ptr->harga << endl;

    tampilBarangRekursif(dataBarang, jumlahBarang, index + 1);
}
```

C. Fitur Tambah Barang

Fitur ini digunakan jika pengguna ingin menambah data barang.

Source Code:

```
//TAMBAH BARANG
void tambahBarang(Barang dataBarang[], int *jumlahBarang){

    system("cls");
    cout << "=== Tambah Data Barang ===\n";

    cout << "Kode Barang   : ";
    cin >> dataBarang[*jumlahBarang].kode;

    cin.ignore();
    cout << "Nama Barang   : ";
    getline(cin, dataBarang[*jumlahBarang].nama);
```

```
    cout << "Ukuran        : ";
    getline(cin, dataBarang[*jumlahBarang].detail.ukuran);

    cout << "Stok          : ";
    cin >> dataBarang[*jumlahBarang].stok;

    cout << "Harga         : ";
    cin >> dataBarang[*jumlahBarang].harga;

    (*jumlahBarang)++;

    cout << "\nData berhasil ditambahkan!\n";
    tekanEnter();
}
```

D. Fitur Lihat Barang

Fitur ini digunakan jika pengguna ingin melihat data barang yang ada.

Source Code:

```
//LIHAT BARANG
void lihatBarang(Barang dataBarang[], int *jumlahBarang){

    system("cls");

    if(*jumlahBarang == 0){
        cout << "Tidak ada data.\n";
    } else {

        cout << fixed << setprecision(0);

        cout <<
"=====================================================================\n";
        cout << left << setw(10) << "Kode"
             << setw(20) << "Nama"
             << setw(10) << "Ukuran"
             << setw(10) << "Stok"
             << setw(15) << "Harga" << endl;
        cout <<
"=====================================================================\n";

        tampilBarangRekursif(dataBarang, jumlahBarang, 0);
```

```
        cout <<
"=====================================================================\n";
    }

    tekanEnter();
}
```

E. Fitur Update Barang

Fitur ini digunakan jika pengguna ingin mengupdate data barang yang sudah di tambahkan sebelumnya.

Source Code:

```
//UPDATE BARANG
void updateBarang(Barang dataBarang[], int *jumlahBarang){

    string kode;

    system("cls");
    cout << "Masukkan kode barang: ";
    cin >> kode;

    int index = cariBarang(dataBarang, jumlahBarang, kode);

    if(index != -1){

        Barang *ptr = &dataBarang[index];

        system("cls");
        cout << "=== Data Barang ===\n";
        cout << "Kode   : " << ptr->kode << endl;
        cout << "Nama   : " << ptr->nama << endl;
        cout << "Ukuran : " << ptr->detail.ukuran << endl;
        cout << "Stok   : " << ptr->stok << endl;
        cout << "Harga  : " << ptr->harga << endl;

        cin.ignore();

        string namaBaru, ukuranBaru, stokBaru, hargaBaru;

        cout << "\n(Kosongkan jika tidak ingin diubah)\n";
```

```
        cout << "Nama baru   : ";
        getline(cin, namaBaru);

        cout << "Ukuran baru : ";
        getline(cin, ukuranBaru);

        cout << "Stok baru   : ";
        getline(cin, stokBaru);

        cout << "Harga baru  : ";
        getline(cin, hargaBaru);

        if(namaBaru != "") ptr->nama = namaBaru;
        if(ukuranBaru != "") ptr->detail.ukuran = ukuranBaru;
        if(stokBaru != "") ptr->stok = stoi(stokBaru);
        if(hargaBaru != "") ptr->harga = stod(hargaBaru);

        cout << "\nData berhasil diupdate!\n";

    } else {
        cout << "Data tidak ditemukan.\n";
    }

    tekanEnter();
}
```

F. Fitur Hapus Barang

Fitur ini digunakan jika pengguna ingin menghapus data barang.

Source Code:

```
 //HAPUS BARANG
void hapusBarang(Barang dataBarang[], int *jumlahBarang){

    string kode;

    system("cls");
    cout << "Masukkan kode barang: ";
    cin >> kode;

    int index = cariBarang(dataBarang, jumlahBarang, kode);

    if(index != -1){
```

```
        for(int i = index; i < *jumlahBarang - 1; i++){
            dataBarang[i] = dataBarang[i+1];
        }

        (*jumlahBarang)--;

        cout << "\nData berhasil dihapus!\n";
    } else {
        cout << "Data tidak ditemukan.\n";
    }

    tekanEnter();
}
```

4. Hasil Output

```
=== LOGIN ===
Username (Nama): ghina
Password (NIM): 054
```

Gambar 4.1 Output Login

```
=== Tambah Data Barang ===
Kode Barang    : K01
Nama Barang    : Cardigan Cable Knit
Ukuran         : ALL SIZE
Stok           : 6
Harga          : 125000

Data berhasil ditambahkan!

Tekan enter untuk lanjut. . .
```

Gambar 4.2 Output Pilihan Menu 1

```
=====================================================================
Kode      Nama                    Ukuran    Stok      Harga
=====================================================================
K01       Cardigan Cable Knit ALL SIZE  6         125000
=====================================================================

Tekan enter untuk lanjut. . .
```

Gambar 4.3 Output Pilihan Menu 2

```
=== Data Barang ===
Kode   : K01
Nama   : Cardigan Cable Knit
Ukuran : ALL SIZE
Stok   : 6
Harga  : 125000

(Kosongkan jika tidak ingin diubah)
Nama baru   :
Ukuran baru :
Stok baru   : 3
Harga baru  :

Data berhasil diupdate!

Tekan enter untuk lanjut. . .
```

Gambar 4.4 Output Pilihan Menu 3

```
Masukkan kode barang: K01

Data berhasil dihapus!

Tekan enter untuk lanjut. . .
```

Gambar 4.5 Output Pilihan Menu 4

```
=== MENU UTAMA ===
1. Tambah Barang
2. Lihat Barang
3. Update Barang
4. Hapus Barang
5. Keluar
Pilih: 5
Terima kasih!
PS C:\praktikum-apl\post-test\post-test-apl-4>
```

Gambar 4.5 Output Pilihan Menu 5

5. Langkah-langkah GIT

5.1 GIT Add

Git add adalah perintah selanjutnya untuk memilih file mana yang akan disimpan perubahannya ke dalam git.

```
PS C:\praktikum-apl> git add .
```

Gambar 5.1.1 Screenshot GIT Add

5.2 GIT Commit

Git commit adalah perintah untuk menyimpan perubahan file yang sudah dipilih sebelumnya (dengan git add) ke dalam riwayat git.

```
PS C:\praktikum-apl> git commit -m "Menyelesaikan kode program"
[main 6e57d72] Menyelesaikan kode program
 2 files changed, 273 insertions(+)
 create mode 100644 post-test/post-test-apl-4/2509106054-GhinaCyntiaRamadhani-PT-4.cpp
 create mode 100644 post-test/post-test-apl-4/2509106054-GhinaCyntiaRamadhani-PT-4.exe
```

Gambar 5.2.1 Screenshot GIT Commit

5.3 GIT Push

Git push adalah perintah untuk mengupload perubahan dari repository lokal ke repository online, seperti github.

```
PS C:\praktikum-apl> git push -u origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 105.51 KiB | 8.79 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/GhinaCyntia29/praktikum-apl
   ba8f0d0..6e57d72  main -> main
branch 'main' set up to track 'origin/main'.
```

Gambar 5.3.1 Screenshot GIT Push